Backends for Frontends (BFF) Pattern

Category: Edge / routing **One-liner:** Instead of one shared backend serving every kind of client, give each client type its own backend tailored to exactly what that client needs.

Origin: Coined by **Sam Newman**, to solve the problem of building backend services for structurally different clients — mobile apps, web applications, and other client-side **frameworks** (ready-made code foundations you build an app on top of, such as React or SwiftUI).

Part 0 — Concepts you need first

What is a "frontend" and a "backend"?

The **frontend** is the part the user sees and interacts with — the mobile app, the website, the smart-TV interface. The **backend** is the server-side software the frontend talks to for data and operations.

A **client** in this chapter means a frontend application. "Client types" means categories of them: mobile, web, TV, watch.

What is over-fetching?

Over-fetching is when a client downloads more data than it actually needs.

Concretely: a mobile app shows a product list with a thumbnail, name, and price. But the API returns the full product object — a 2,000-word description, fifteen high-resolution image **URLs** (web addresses), full specifications, the seller's history. The app downloads all of it and displays three fields.

Why it hurts:

- **Bandwidth** (how much data a connection can carry, and how much you use) — on a **metered** mobile plan (one that charges per megabyte), this is the user's money
- **Latency** (the time a request takes from start to finish) — larger **payloads** (the actual data carried in a request or response) take longer over slow connections
- **Battery and CPU** (the processor that executes code) — the phone must **parse** (read and convert into usable objects) **JSON** (JavaScript Object Notation, a common text format for data) it discards
- **Memory** — constrained devices hold data they never use

What is an API?

An **API** (Application Programming Interface) is the agreed contract for how one piece of software asks another for something: what you can request, what parameters you supply, what shape comes back.

When this chapter says "one uniform set of APIs for all frontends," it means every client — however different — gets the same **endpoints** (specific URLs a server answers, like `GET /products/42`) returning the same shapes.

What is aggregation?

Combining data from several sources into one response. If a screen needs product details, reviews, and stock levels from three services, **aggregation** means one component calls all three and returns a single merged result, so the client makes one request instead of three.

Why round trips matter so much on mobile

A **round trip** is one request out and one response back. On a fast office network it might take 5-20ms. On a mobile network it can be **100-300ms or worse**.

Six round trips on a desktop: barely noticeable. Six round trips on mobile: potentially two seconds of nothing happening. This single fact drives most of this chapter.

Part 1 — The core idea, in plain terms

Imagine a company with three digital platforms:

- **A detailed website** — for users on computers, offering extensive information
- **A mobile app** — quick, essential services for on-the-go use
- **A smartwatch app** — core functions only, in a very compact form

These are genuinely different products with genuinely different needs. The website can display a thousand words; the watch can display about eight.

Traditionally, one backend serves all three. The course's analogy: BFF means giving **each section of a diverse restaurant its own kitchen**, rather than forcing one kitchen to cook for wildly different menus.

The reason this matters: the backend complexity appropriate for the website **overwhelms** the simpler needs of mobile and watch. The website's needs win by default, because it's usually the biggest and oldest client, and everyone else gets its leftovers.

Netflix does exactly this — web browsers, mobile apps, and smart TVs each get their own backend service, because each device has different capabilities and interfaces.

Part 2 — The problem in detail

One-size-fits-all

In traditional backend models the backend is a **monolithic** entity (one large application built and deployed as a single unit) serving multiple frontends, exposing **one uniform set of APIs regardless of each client's unique requirements**.

Consider a desktop app and a mobile app hitting the same API:

	Desktop	Mobile
Processing power	High	Limited
Display	Large	Small
Network	Usually fast, unmetered	Possibly slow, possibly metered

The course asks the right questions: should the mobile app receive the same amount of data as the desktop app? Is it efficient to process all that data in a mobile environment? **Not quite.**

Performance inefficiency and over-fetching

This leads directly to over-fetching, which causes performance issues, longer loading times, and increased bandwidth usage.

Worth being concrete about the cost. A product list screen:

- **Shared API response:** 250 KB (full descriptions, high-res image URLs, full specs)
- **What mobile displays:** name, price, thumbnail — about 4 KB of actual information

The user paid for 250 KB and used 4 KB. On a slow connection that's several seconds of waiting for data that gets thrown away.

Complexity in code management

When a single **codebase** (the full body of source code for one application) serves multiple frontends it becomes **bloated and complex** — harder to maintain, troubleshoot, and update. The more complex the system, the more time and resources it consumes, which could have gone to productive work.

In practice this shows up as endpoints crowded with **conditionals** (`if`/`else` branches):

```
if (clientType.equals("mobile")) {
    // trim the description
} else if (clientType.equals("tv")) {
    // use high-res images
} else if (clientType.equals("watch")) {
    // just the title
}
```

Every client's needs tangled into one method that nobody can change safely.

Lack of flexibility and scalability

Traditional backends have a rigid structure that makes it hard to innovate and scale. The course gives two symptoms that should feel obviously wrong:

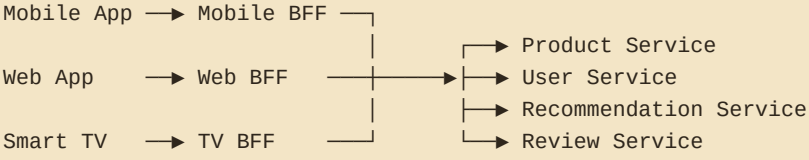
1. **Sifting through a monolithic codebase to change one small feature for one specific frontend**
2. **Scaling the entire backend system because one frontend is experiencing high traffic**

Both are overkill — that's exactly the signal you need BFF.

> *Why symptom 2 matters:* suppose a TV advert drives a spike in mobile traffic. With a shared backend you scale the whole thing — including the capacity serving web and internal tools that aren't busy at all. You pay for capacity you don't need because the parts can't be scaled separately.

Part 3 — Architecture

Three layers:



1. Frontend applications — the user-facing applications: desktop, mobile, web. *The course's analogy: the customers ordering from a menu.*

2. BFFs — each paired with one specific frontend, catering to its unique requirements. *The chefs, preparing each dish to a customer's preference.*

3. Microservices — the underlying services BFFs call to fetch data. (**Microservices** are small, independently deployable services, each owning one business capability and its data.) *The kitchen staff providing the raw materials.*

The important structural point: **the microservices are shared; only the BFF layer is duplicated.** There is still exactly one Product Service holding product truth — the **source of truth** (the one place whose copy of the data is authoritative). BFFs don't own data — they reshape it.

What a BFF actually does

Tailored data delivery. Each BFF ensures its frontend receives *just* the data it needs. It acts as a **filter**, ensuring the frontend isn't overwhelmed with unnecessary information.

Streamlining communication. The BFF talks to multiple microservices, consolidates the data, and presents it to the frontend in one streamlined shape. The course calls it an **interpreter**, translating many languages into one the frontend understands.

Handling client-specific logic. Each BFF absorbs logic specific to its frontend, relieving the client of complex computation — a **personal assistant** doing work so the client's job is easier.

> *Why moving logic server-side matters especially for mobile:* you cannot force users to update a mobile app. If display logic lives in the app, fixing it means shipping a release, waiting for app store review, and waiting for users to update — some never will. If it lives in the BFF, you fix it and **deploy** (release the new version to the live servers) in minutes.

Part 4 — The benefits, explained

The course lists seventeen. Grouped by theme, with the reasoning behind each.

Fit and user experience

Diverse frontend needs. A mobile app, a web app, and a smart TV app have genuinely different requirements. BFF acknowledges that diversity instead of pretending it away.

Enhanced user experience. By catering to each device's specific needs and capabilities, the user gets the best possible experience on whatever they're using.

Localization and internationalization. Different frontends may serve users in different regions with different languages and cultural contexts. A BFF can serve localized content and region-relevant features.

Performance

Performance optimization. Mobile devices may be on slower networks with less processing power than a desktop. A BFF lets you optimise the data and interactions per device for fast, responsive apps.

Reduced bandwidth usage. Especially important for mobile — the BFF sends only what that specific frontend requires.

Scalability. (The ability to handle more load by adding resources.) Each frontend has its own backend, so you can scale parts of the system independently based on their own demand. A mobile traffic spike scales only the Mobile BFF.

Simplicity for frontend teams

Simplified frontend logic. Frontends no longer deal with the complexity of interacting with multiple microservices. They have a single, tailored backend that aggregates and processes data for them.

Consistency. A BFF can provide a **consistent API even while the underlying microservices are constantly changing**. This is a stability layer for frontend developers — the backend team can reorganise services without breaking every app.

Versioning control. Different frontend applications may be at different versions requiring different backend logic. A BFF handles versions gracefully, so each frontend gets logic appropriate to its version. *(Again most acute on mobile, where old versions persist in the wild.)*

Team and delivery

Agility. Teams work independently on their own BFF, allowing faster development and deployment cycles. The mobile team doesn't queue behind the web team.

Easier maintenance and upgrades. You can update the mobile backend without affecting the web backend, and vice versa.

Cross-functional teams. BFF promotes teams that own **end-to-end functionality for one user interface** — frontend and its backend together. Better collaboration, faster delivery.

Capability

Security. Each client type can have its own security considerations. A mobile app may need different **authentication** mechanisms (ways of proving who the user is, e.g. cookies vs. tokens) than a web app.

Error handling and resilience (the ability to keep working, perhaps in reduced form, when parts fail). You can implement error handling specific to each frontend. A web application might handle certain errors gracefully where a mobile app cannot.

Facilitates A/B testing (showing two variants to different user groups and comparing the results). You can test new features or changes on one client type — say the mobile app — **without affecting the others**.

Feature flagging. (A **feature flag** is a configuration switch that turns behaviour on or off without deploying new code.) Enable or disable features for specific frontends at the backend level, without affecting others.

Handling legacy systems. During a monolith-to-microservices transition, a BFF acts as a **bridge**, letting new microservices-based frontends interact with **legacy systems** (old systems still running because the business depends on them). *(See Strangler Fig.)*

Part 5 — Worked example: e-commerce product details

Three components: one shared **Product Service**, a **Web BFF** (extensive detail), and a **Mobile BFF** (concise, to save data and improve load times).

1. Product Service — the real backend

```
@RestController
@RequestMapping("/products")
public class ProductServiceController {

    @Autowired
    private ProductRepository productRepository;

    @GetMapping("/{productId}")
    public Product getProductById(@PathVariable String productId) {
        return productRepository.findById(productId)
                .orElseThrow(() -> new ProductNotFoundException(productId));
    }
}
```

Reading it: (the code uses **Spring**, the dominant Java web framework; words starting with `@` are **annotations** — labels that tell the framework how to treat a class or method)

- `@RestController` marks this a web-request handler returning data (not **HTML** pages — HTML being the markup language browsers render)

- `@Autowired` asks Spring to supply ("inject") the `ProductRepository` object automatically; a **repository** is the class that reads and writes this data in the database
- `@RequestMapping("/products")` — every endpoint here starts with `/products`
- `@GetMapping("/{productId}")` — handles `GET /products/123`; `{productId}` is a placeholder
- `@PathVariable` pulls that value out of the URL into the method parameter
- `orElseThrow` — if no product exists, raise a clear error rather than returning `null` (Java's "no object here" value)

This service owns the **full, detailed** `Product` and does the heavy lifting. It knows nothing about mobile or web.

2. Web BFF — the detailed view

```
@RestController
@RequestMapping("/web/products")
public class WebProductController {

    @Autowired
    private ProductServiceClient productServiceClient;

    @GetMapping("/{productId}")
    public ProductDetails getProductDetails(@PathVariable String productId) {
        Product product = productServiceClient.getProductById(productId);
        return convertToWebProductDetails(product);
    }

    private ProductDetails convertToWebProductDetails(Product product) {
        // Convert to a detailed view suitable for web:
        // richer descriptions, full-size images, etc.
    }
}
```

Note the URL prefix `/web/products` — each BFF has its own namespace, so clients can't accidentally call the wrong one.

`ProductServiceClient` is a **Feign client** or any **HTTP client** (code that sends web requests using HTTP, the protocol of the web) used to talk to the Product Service.

> *Beginner note — what's a Feign client?* A library that lets you call another service by declaring an interface, and generates the HTTP-calling code for you. You call `getProductById(id)` like a normal method and it makes the network request behind the scenes.

3. Mobile BFF — the concise view

```java
@RestController
@RequestMapping("/mobile/products")
public class MobileProductController {

    @Autowired
    private ProductServiceClient productServiceClient;

    @GetMapping("/{productId}")
    public SimpleProductDetails getProductDetails(@PathVariable String productId) {
        Product product = productServiceClient.getProductById(productId);
        return convertToMobileProductDetails(product);
    }

    private SimpleProductDetails convertToMobileProductDetails(Product product) {
        // Convert to a concise view suitable for mobile:
        // stripped-down descriptions, smaller images, etc.
    }
}
```

The structural point — read this part twice

Compare the two BFFs. They are **nearly identical**:

1. Both call the **same** `productServiceClient.getProductById(productId)`
2. Both receive the **same** full `Product` object
3. They differ **only in the transformation step** and therefore the return type:
    - Web returns `ProductDetails` — rich descriptions, full images
    - Mobile returns `SimpleProductDetails` — stripped descriptions, smaller images

A BFF is a projection layer, not a second source of truth. (A **projection** is a reshaped view of data that lives elsewhere.) There is one Product Service holding one truth about products. The BFFs each project a different view of it. If you ever find a BFF *storing* data or deciding business rules about what a product *is*, it has stopped being a BFF.

> **How it works in practice**

- The **Product Service** handles all the heavy lifting related to product data
- The **Web BFF** provides a detailed view suitable for web users
- The **Mobile BFF** provides a concise view suitable for mobile users

Each frontend gets exactly what it needs, and the backend logic is tailored to give the best possible experience.

Part 6 — Problems with BFF, and how to handle them

1. Code duplication

Problem: Since each frontend has its own BFF, there may be **duplication across BFFs**, especially where they share common functionality — authentication handling, error formats, logging. **Solution:** Use **shared libraries** (reusable code packages that several projects import) for common functionality, or create **internal microservices** that multiple BFFs consume.

2. Increased maintenance

Problem: More BFFs mean **more codebases to maintain**, increasing operational overhead. Three BFFs means three **deployment pipelines** (automated build-test-release sequences), three sets of **dependencies** (external libraries the code relies on) to patch. **Solution:** Adopt good **DevOps** practices (development and operations working together, with heavy automation of releases and monitoring), automate testing and deployment, and ensure proper monitoring to keep maintenance manageable.

3. Coordination overhead

Problem: Coordinating changes between frontends and their BFFs becomes challenging in large teams. **Solution:** Establish **clear contracts** between frontends and BFFs, and use **Consumer-Driven Contract Testing** to ensure compatibility.

> *Beginner note — consumer-driven contract testing:* the consumer (the app) writes down what it expects from the BFF as an executable test. The BFF's build runs those tests. If the BFF changes something the app depends on, the BFF's build fails — before it reaches production, rather than after.

Performance implications

Network latency. BFF adds an additional network **hop** (one more trip between machines), contributing to latency (see System Design Fundamentals). *Solution:* optimise network paths, use efficient **protocols** (agreed rules for how data is exchanged, e.g. HTTP/2 or gRPC), and ensure **BFFs are geographically close to the frontends they serve**.

Resource utilization. Each BFF may require its own set of resources. *Solution:* optimise BFFs for performance, use **serverless** (cloud functions where the provider runs and scales the servers for you and you pay per call) where appropriate, and scale on demand.

Caching challenges. Caching gets complex when multiple frontends have different requirements — the same underlying data cached in three different shapes. *Solution:* intelligent **caching** (keeping copies of recent answers to avoid refetching) at both BFF and frontend layers, and effective use of `Cache-Control` **headers** (HTTP metadata telling browsers and proxies how long a response may be reused).

Load on backend services. Multiple BFFs can increase load on the shared backend services. Three BFFs each fetching the same product means three calls where there was one. *Solution:* ensure backend services are scalable, monitor their performance, and implement **rate limiting** (capping how many requests a caller may make per second) if necessary.

> ⚠️ **The duplication trap is the one that actually bites.** Three BFFs slowly drifting apart on auth handling or error semantics is how "tailored" quietly becomes "inconsistent" — and users hit a bug on mobile that doesn't exist on web. Shared libraries are not optional at scale.

Part 7 — System design example: Netflix

Netflix serves smartphones, tablets, smart TVs, laptops, and gaming consoles. Each has distinct characteristics, user interactions, and performance considerations.

Scenario: viewing movie details — the user selects a movie and sees title, description, genre, rating, cast, recommendations, and user reviews.

Different devices prioritise differently:

Device	Priority
Smartphones	Quick loading, less data usage
Smart TVs	Rich visuals, detailed information, high-quality images
Web browsers	Balance between performance and rich content

Three BFFs: **Mobile BFF** (phones and tablets), **TV BFF** (smart TVs), **Web BFF** (browsers).

Mobile BFF

Serves concise movie details, smaller images, and **a subset of recommendations** to optimise for speed and data usage.

- **Authentication** — ensures the user is logged in and has a valid subscription
- **Routing** — directs the request to the appropriate microservices (Movie Details Service, Recommendation Service)
- **Aggregation** — combines data from those services and tailors it for mobile
- **Transformation** — reduces image sizes, shortens descriptions, formats data for quick mobile rendering

TV BFF

Serves rich visuals, detailed movie information, and a comprehensive list of recommendations.

- **Authentication & authorization** — checks not just login status but **whether the device is authorized** (allowed — as opposed to authenticated, which only checks identity) **to play high-definition content**
- **Routing & aggregation** — similar to Mobile BFF, but optimised for TV capabilities and network conditions
- **Transformation** — high-resolution images, full cast lists, elaborate descriptions

Web BFF

Balances performance and content richness.

- **Authentication** — standard login check
- **Routing & aggregation** — tailored for web browsers
- **Transformation** — content optimised for a wide range of browsers and screen sizes

The benefits realised

- **User experience** — each device gets an optimised, tailored experience
- **Performance** — data streamlined per device type, improving load times
- **Maintainability** — frontend teams work closely with their own BFF, leading to faster iterations

The challenges realised

- **Maintenance** — each BFF must be maintained and updated, creating overhead
- **Consistency** — ensuring consistent behaviour across all BFFs is genuinely challenging

> **The TV BFF's extra HD-entitlement check is the clearest single argument for BFF over one shared gateway.** That rule is meaningless for phones and browsers. In a shared gateway it would sit behind a device-type conditional, polluting a component every team must touch. In the TV BFF it's simply that BFF's business.

Part 8 — BFF vs. API Gateway

	API Gateway	BFF
How many	One shared entry point	One per client type
Optimised for	All clients generically	One specific client
Client-specific logic	Pollutes it	Belongs there
Main risk	Bottleneck / distributed monolith	Duplication and drift
Owned by	Usually a platform team (builds shared infrastructure)	Usually the frontend team

They are not mutually exclusive. A very common production setup is a thin shared gateway handling **TLS termination** (decrypting incoming HTTPS traffic at the edge so services behind it receive plain requests), coarse authentication, and rate limiting, with BFFs sitting behind it. The gateway does what's identical for everyone; the BFFs do what differs.

Think of BFF as API Gateway specialised per client, and note the ownership difference — a BFF being owned by the team that owns the frontend is a large part of why it delivers agility.

Part 9 — Common beginner misconceptions

"Each BFF needs its own database." No. BFFs are generally **stateless** (they keep no data of their own between requests). They call shared microservices and reshape responses. A BFF with its own database has usually turned into a microservice by accident.

"BFF means one backend per screen." No — per **client type**. All mobile screens share the Mobile BFF.

"You need BFF as soon as you have a mobile app." Only when needs genuinely diverge. If mobile and web want nearly the same data, a shared gateway is simpler. Adopt BFF when you're adding device-conditionals to shared code.

"GraphQL makes BFF obsolete." GraphQL is a query language for APIs, where a single endpoint accepts queries naming the exact fields wanted. GraphQL lets clients request exactly the fields they want, solving over-fetching without separate backends — a genuine alternative for *shaping*. But it doesn't solve client-specific *logic* (the HD-entitlement check) or independent scaling and ownership. Some teams use both: a GraphQL layer per client type.

"BFFs make the system faster." Not inherently — they add a hop. They make it faster *for the client* by cutting round trips and payload size, which dominates on mobile networks.

Part 10 — Check your understanding

1. What is over-fetching, and why does it matter more on mobile than desktop?
2. In the worked example, what is the **only** thing that differs between Web BFF and Mobile BFF?
3. Why is it wrong for a BFF to own a database?
4. What's the strongest argument for the TV BFF over a shared gateway?
5. Name the main cost of BFF and its standard mitigation.
6. When should you *not* use BFF?

(Answers: 1 — downloading more data than needed; mobile has metered data, slow networks, limited CPU/memory; 2 — the transformation step and its return type, both call the same service for the same object; 3 — BFFs are projection layers, not sources of truth — one service owns each piece of data;

4 — the HD-entitlement authorization check is meaningless to other clients and would pollute shared code; 5 — code duplication across BFFs, mitigated by shared libraries and internal services; 6 — when clients' needs are nearly identical, or the team is too small to maintain several backends.)

Part 11 — Interview soundbite (30 seconds)

> "BFF, from Sam Newman, means giving each client type its own backend instead of sharing one. The problem it solves is over-fetching and one-size-fits-all APIs — a mobile app on a metered connection shouldn't receive a payload designed for a desktop, and it shouldn't need six round trips on a slow network to build one screen. Each BFF calls the same underlying microservices but aggregates and transforms for its client: mobile gets small images and short descriptions, TV gets high-res and full cast lists. Crucially the BFF is a projection layer, not a source of truth — the microservices still own the data. Netflix does this across phones, TVs, and browsers, and their TV BFF has an extra authorization check for HD entitlement, which is a good example of client-specific logic that would pollute a shared gateway. It also lets frontend teams own their backend and ship independently, and lets you scale per client. The costs are code duplication across BFFs, more codebases to maintain, an extra network hop, and keeping behaviour consistent between them — so shared libraries and consumer-driven contract testing matter. It's essentially API Gateway specialised per client rather than one shared gateway, and the two are often combined."

Part 12 — Implementation

A. Runnable demo

Full source: `demos/BffDemo.java`

```
cd demos && ./gradlew runCh04
```

The essential structure — three BFFs, one **upstream** service (the service a component calls to get its data), three projections:

```
// Mobile: trim the copy, shrink the image
new MobileProductDetails(
    p.id(), p.name(), p.price(),
    truncate(p.longDescription(), 120),
    p.hiResImages().get(0).replace("4k", "thumb"));

// Watch: two fields, nothing else
new WatchProductDetails(truncate(p.name(), 20), p.price());
```

B. What you should see

```
=== 1. The over-fetching problem ===
  shared API payload:   3,525 bytes
  >> a watch showing name + price downloads ALL of that

=== 2. Each BFF projects the same object differently ===
  Web BFF:      3,525 bytes  (100% of shared)
  Mobile BFF:     251 bytes  (  7% of shared)
  Watch BFF:       31 bytes  (  0% of shared)
  >> mobile saves ~3274 bytes PER PRODUCT

=== 3. All BFFs call the SAME upstream service ===
  ProductService invocations so far: 4
  >> BFFs are projection layers - one source of truth remains

=== 4. Client-specific logic belongs in its own BFF ===
  certified TV:     Full HD payload ... with 3 4K images
  uncertified TV:   SD-only payload ...
```

Mobile receives 7% of the shared payload. Multiply by a 50-item product list and it's the difference between a screen that loads instantly and one that visibly stalls on a train.

Section 3 is the conceptual check: every BFF called the *same* `ProductService`. **They add no data and own no data.** A BFF that starts storing things has stopped being a BFF.

C. Production — a BFF is just a small service

There's no special framework; a BFF is an ordinary application whose job is aggregation and shaping. Below, `CompletableFuture.supplyAsync` runs each call on a background thread so the three calls happen in parallel, `.exceptionally(...)` supplies a default if a call fails, and `join()` waits for the results.

```java
@RestController
@RequestMapping("/mobile/v1")
public class MobileBffController {

    private final ProductClient products;
    private final ReviewClient reviews;
    private final InventoryClient inventory;

    @GetMapping("/products/{id}")
    public MobileProductView get(@PathVariable String id) {
        // Fan out in parallel - the BFF exists to save the client round trips
        var pf = CompletableFuture.supplyAsync(() -> products.byId(id));
        var rf = CompletableFuture.supplyAsync(() -> reviews.summaryFor(id))
                    .exceptionally(e -> ReviewSummary.empty());     // degrade, don't fail
        var inf = CompletableFuture.supplyAsync(() -> inventory.stockFor(id))
                    .exceptionally(e -> Stock.unknown());

        CompletableFuture.allOf(pf, rf, inf).join();
        return MobileProductView.from(pf.join(), rf.join(), inf.join());
    }
}
```

GraphQL as an alternative. It solves the *shaping* half of the problem — clients request exactly the fields they need, so one endpoint serves all clients without over-fetching:

```
query MobileProduct($id: ID!) {
  product(id: $id) { id name price thumbnail }     # mobile asks for four fields
}
```

But it does **not** solve client-specific *logic* (the TV HD-entitlement check), independent scaling, or team ownership. A common production setup is **one GraphQL BFF per client type** — shaping from GraphQL, isolation from BFF.

Shared code across BFFs — the mitigation for the duplication problem (**tracing** below means recording a request's path across services so it can be followed end to end):

```
bff-common/          <- auth, error format, tracing, resilience config
  ├─ mobile-bff/
  ├─ web-bff/
  └─ tv-bff/
```

D. Implementation pitfalls

1. Giving a BFF its own database. The single most common mistake. BFFs are **stateless projection layers**. **Persistent state** (data saved so it survives restarts) means you've accidentally created a microservice with duplicated ownership of data.

2. Sequential upstream calls. A BFF exists to cut client round trips. Making three sequential server-side calls just moves the latency rather than removing it. Fan out with `CompletableFuture.allOf`.

3. No fallback on secondary data. If reviews are down, a product page should still render. Note the `.exceptionally(...)` on the review and inventory futures but *not* on the product itself — the primary resource legitimately fails the request; supporting data must not.

4. Copy-pasting auth into every BFF. This is how three BFFs drift into three subtly different security behaviours. Extract to `bff-common` from the start.

5. One BFF per screen. Per **client type**, not per screen. Twenty BFFs is twenty deployment pipelines for no benefit.

6. Business rules migrating into the BFF. "Is this order eligible for a refund?" belongs in the order service, where every client gets the same answer. Only **presentation** decisions belong in a BFF.

7. Versioning by breaking changes. (A **breaking change** is one that makes existing clients stop working, such as removing or renaming a field.) Old mobile app versions live in the wild for years. Version the BFF path (`/mobile/v1`, `/mobile/v2`) and keep old versions running — being able to do this is a major reason BFF exists.

Related patterns

- **API Gateway** — the shared-entry-point alternative, and frequent companion
- **Strangler Fig** — a BFF can bridge new frontends to legacy systems mid-migration
- **Service Discovery** — how BFFs locate the services they aggregate
- **Circuit Breaker** — a BFF calling several services needs protection when one fails